

Neural networks — Lab 1: from a numpy perceptron to a convolutional network

In this lab you build a two-layer neural network by hand in numpy, train it to tell birds from dogs, and then let Keras do the same job with a convolutional network. Along the way you meet the ideas that Lecture 2 builds on: the forward and backward pass, the loss curve, overfitting, dropout and early stopping. At the end you apply what you learned to the MNIST digits.

Outline

- Part A. A two-layer network in numpy (bird vs dog, 32×32 images flattened to 3072 numbers)
- Part B. The same problem with a convolutional network in Keras, with and without dropout
- Part C. A small data set: 19 face images, happy or not happy, and transfer learning
- Part D. MNIST handwritten digits (task)

Cells marked **Task** contain `# your code here` . Everything else runs as given.

Setup

On Colab, choose *Runtime* → *Change runtime type* → *GPU* before you start; the Keras parts run faster.

The cell below imports Keras. Keras 3 can run on TensorFlow (Colab, most local installs) or on PyTorch; if TensorFlow is missing we switch to the PyTorch backend, the rest of the notebook is identical. It then finds the training images: on Colab it clones the course repository, otherwise it looks for a `train` folder next to the notebook.

```
In [95]: import os, sys
try:
    import tensorflow # present on Colab and most local installs
except ImportError:
    os.environ["KERAS_BACKEND"] = "torch" # Keras 3 also runs on PyTorch
import keras
from keras import layers
print("Keras", keras.__version__, "backend:", keras.backend.backend())

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```

from sklearn.model_selection import StratifiedKFold, cross_val_score

keras.utils.set_random_seed(0) # numpy, python and backend seeds, so re

# --- locate the training images -----
if not Path("Bern02/Labs/Neural_Networks/train").exists() and "google.col
    !git clone https://github.com/luchem/Bern02.git --depth=1
candidates = ["Bern02/Labs/Neural_Networks/train", "train",
              "/home/user/workspace/nn_data/bern02/Labs/Neural_Networks/t
DATA = next(Path(p) for p in candidates if Path(p).exists())
print("data folder:", DATA.resolve())

```

Keras 3.15.1 backend: tensorflow

data folder: /Users/labergnilsson/Documents/BERN_02/Exercise_NN_1_and_2/train

Part A. A two-layer network in numpy

An artificial neuron has six parts, from input to output:

1. Input values $x_1, \dots, x_D$ (here: the pixel values of an image).
2. One weight w_i per input.
3. A weighted sum $z = \sum_{i=1}^D w_i x_i + b$. The constant b is the *bias*; it shifts the threshold.
4. An *activation function* $g(z)$ that turns the sum into the neuron's output. A hard step (0 if $z \leq 0$, 1 if $z > 0$) is not differentiable, so one uses smooth or piecewise-linear functions instead: the logistic (sigmoid) function $\sigma(z) = 1/(1 + e^{-z})$, or the rectified linear unit $\text{ReLU}(z) = \max(0, z)$.
5. The output $a = g(z)$.
6. Connections to the next layer, where a becomes an input.

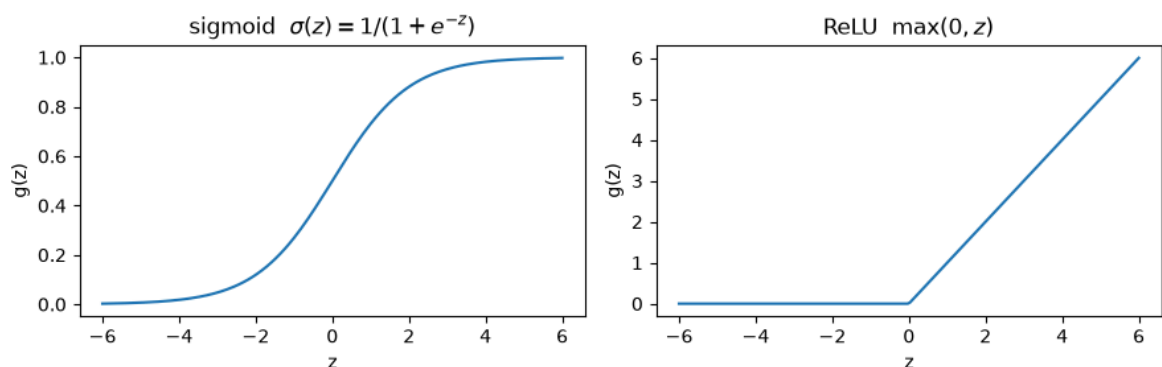
A *layer* is a set of such neurons that share the same inputs; a network stacks layers.

The plot shows the two activation functions we use.

```

In [96]: z = np.linspace(-6, 6, 200)
fig, ax = plt.subplots(1, 2, figsize=(9, 3))
ax[0].plot(z, 1/(1 + np.exp(-z))); ax[0].set_title(r'sigmoid $\sigma(z)$')
ax[1].plot(z, np.maximum(0, z)); ax[1].set_title(r'ReLU $\max(0, z)$')
for a in ax:
    a.set_xlabel('z'); a.set_ylabel('g(z)')
plt.tight_layout()

```



Loading the data

The folder `train` holds 1200 bird and 1200 dog images of 64×64 pixels. We shrink them to 32×32 pixels (three colour channels), scale the pixel values to $[0, 1]$, and use label 0 for bird and 1 for dog.

```
In [97]: IMAGE_SIZE = 32

def load_folder(folder, size=IMAGE_SIZE):
    "Load every jpg in `folder` as an RGB array of shape (size, size, 3)"
    files = sorted(Path(folder).glob("*.jpg"))
    return np.array([np.asarray(Image.open(p).convert("RGB").resize((size, size),
                                                                    for p in files], dtype=np.float32) / 255.0

birds = load_folder(DATA / "bird")
dogs = load_folder(DATA / "dog")
X = np.concatenate([birds, dogs]) # (N, 32, 32, 3)
y = np.array([0]*len(birds) + [1]*len(dogs)) # 0 = bird, 1 = dog
print("images:", X.shape, " labels:", y.shape, " birds:", len(birds), " d

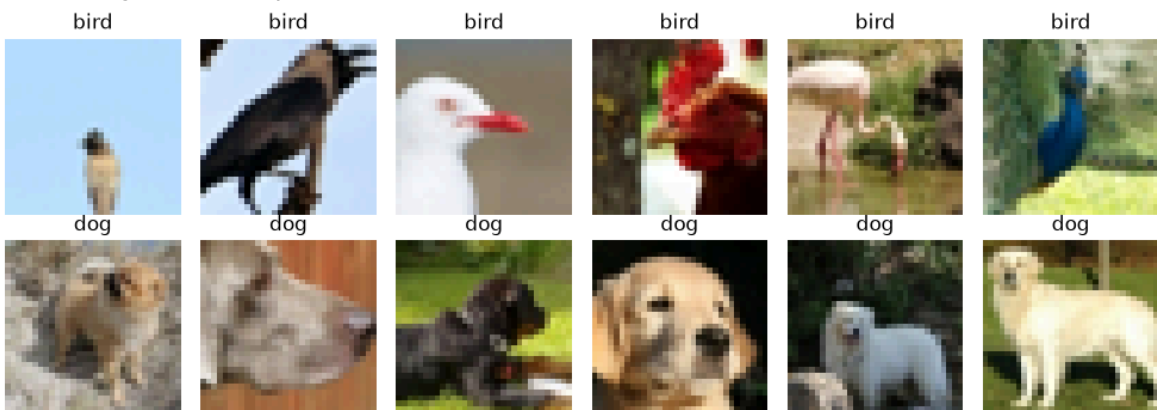
images: (2400, 32, 32, 3) labels: (2400,) birds: 1200 dogs: 1200
```

Looking at the data

How many images do we have in total, and what is the shape of one image? Look at a few: at 32×32 pixels the pictures are coarse, but you can still tell a bird from a dog. Whether the network can is the question of this lab.

```
In [98]: fig, ax = plt.subplots(2, 6, figsize=(10, 3.6))
for i, a in enumerate(ax[0]):
    a.imshow(birds[i]); a.set_title("bird"); a.axis("off")
for i, a in enumerate(ax[1]):
    a.imshow(dogs[i]); a.set_title("dog"); a.axis("off")
plt.tight_layout()
print("one image has shape", X[0].shape, "=", X[0].size, "numbers")
```

one image has shape (32, 32, 3) = 3072 numbers



Train/test split and flattening

We hold out 20 % of the images as a *test* set that the network never sees during training. Only the test accuracy tells us whether the network learned something

general or just memorised the training pictures. `train_test_split` with `stratify=y` keeps the bird/dog ratio equal in both parts.

The two-layer network connects every pixel to every hidden neuron, so the position of a pixel carries no special meaning. We therefore *flatten* each image to a vector of $32 \cdot 32 \cdot 3 = 3072$ numbers. Following the usual convention for hand-written networks, the data matrix has one **column** per image: shape `(3072, m)`.

```
In [59]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
Xf_train = X_train.reshape(len(X_train), -1).T.astype(np.float64) # (30
Xf_test  = X_test.reshape(len(X_test), -1).T.astype(np.float64)   # (30
Y_train  = y_train.reshape(1, -1)                                # (1, m_train)
Y_test   = y_test.reshape(1, -1)
print("train:", Xf_train.shape, Y_train.shape, " test:", Xf_test.shape,
train: (3072, 1920) (1, 1920) test: (3072, 480) (1, 480)
```

The network and its equations

We build a network with one hidden layer of n_h ReLU neurons and one sigmoid output neuron. With the data matrix X of shape (n_x, m) and labels Y of shape $(1, m)$, the **forward pass** is

$$Z_1 = W_1 X + b_1, \quad A_1 = \text{ReLU}(Z_1), \quad Z_2 = W_2 A_1 + b_2, \quad A_2 = \sigma(Z_2).$$

A_2 is the predicted probability that each image is a dog. The **loss** is the binary cross-entropy, averaged over the m images,

$$J = -\frac{1}{m} \sum_{i=1}^m [y_i \log a_i + (1 - y_i) \log(1 - a_i)].$$

The **backward pass** applies the chain rule from the output back to the input. For the sigmoid output with cross-entropy loss the first step is remarkably simple,

$$dZ_2 = A_2 - Y, \quad dW_2 = \frac{1}{m} dZ_2 A_1^T, \quad db_2 = \frac{1}{m} \sum_i dZ_2,$$

and for the hidden layer

$$dA_1 = W_2^T dZ_2, \quad dZ_1 = dA_1 \odot 1[Z_1 > 0], \quad dW_1 = \frac{1}{m} dZ_1 X^T, \quad db_1 = \frac{1}{m}$$

Here dW is short for $\partial J / \partial W$ and $\odot$ is element-wise multiplication. Gradient descent then updates every parameter as $W \leftarrow W - \eta dW$ with learning rate η . The code below implements exactly these lines.

```
In [60]: def initialize_parameters(n_x, n_h, n_y):
"Random small weights, zero biases. Shapes: W1 (n_h, n_x), b1 (n_h, 1)
W1 = np.random.randn(n_h, n_x) * 0.01
b1 = np.zeros((n_h, 1))
W2 = np.random.randn(n_y, n_h) * 0.01
```

```
b2 = np.zeros((n_y, 1))
return {"W1": W1, "b1": b1, "W2": W2, "b2": b2}
```

```
In [99]: def sigmoid(Z):
        return 1 / (1 + np.exp(-Z))

        def relu(Z):
            return np.maximum(0, Z)

        def forward(X, p):
            "Forward pass. Returns the output A2 and a cache of intermediate values"
            Z1 = p["W1"] @ X + p["b1"]
            A1 = relu(Z1)
            Z2 = p["W2"] @ A1 + p["b2"]
            A2 = sigmoid(Z2)
            return A2, (X, Z1, A1, A2)
```

```
In [100]: def compute_cost(A2, Y):
        "Binary cross-entropy averaged over the m examples."
        m = Y.shape[1]
        A2 = np.clip(A2, 1e-12, 1 - 1e-12) # avoid log(0)
        return float(-np.sum(Y*np.log(A2) + (1 - Y)*np.log(1 - A2)) / m)
```

Training is a loop of three steps:

1. **Forward pass:** the data flows through the network and produces the outputs A_2 .
2. **Loss:** compare A_2 with the true labels.
3. **Backward pass:** compute the gradient of the loss with respect to every weight and bias, then move each parameter a small step against its gradient.

Backpropagation is nothing more than the chain rule organised so that each layer reuses what the layer after it already computed.

```
In [101]: def backward(p, cache, Y):
        "Backward pass: gradients of the cost with respect to W1, b1, W2, b2."
        X, Z1, A1, A2 = cache
        m = X.shape[1]
        dZ2 = A2 - Y # sigmoid + cross-entropy
        dW2 = dZ2 @ A1.T / m
        db2 = dZ2.sum(axis=1, keepdims=True) / m
        dA1 = p["W2"].T @ dZ2
        dZ1 = dA1 * (Z1 > 0) # derivative of ReLU
        dW1 = dZ1 @ X.T / m
        db1 = dZ1.sum(axis=1, keepdims=True) / m
        return {"dW1": dW1, "db1": db1, "dW2": dW2, "db2": db2}
```

```
In [102]: def update_parameters(p, grads, learning_rate):
        "One gradient-descent step on every parameter."
        for name in ("W1", "b1", "W2", "b2"):
            p[name] = p[name] - learning_rate * grads["d" + name]
        return p
```

The output A_2 is a probability. To turn it into a class we compare it with 0.5: `(A2 > 0.5)` gives `True` for dog and `False` for bird, and `astype(int)` makes that 1

and 0. (Older versions of this notebook used `np.floor(A2 + 0.5)`, which does the same but hides the threshold.)

```
In [103... def predict(X, p):
    "Class labels (0 = bird, 1 = dog) for the images in X."
    A2, _ = forward(X, p)
    return (A2 > 0.5).astype(int)

def accuracy(X, Y, p):
    return float(np.mean(predict(X, p) == Y))
```

Training loop

`two_layer_model` puts the pieces together. Two details:

- The learning rate is multiplied by 0.999 after every iteration. This is a *learning-rate schedule*: large steps early, smaller steps as we approach a minimum. Lecture 2 explains why this helps.
- Every `record_every` iterations we store the cost and the train and test accuracy so that we can plot them afterwards.

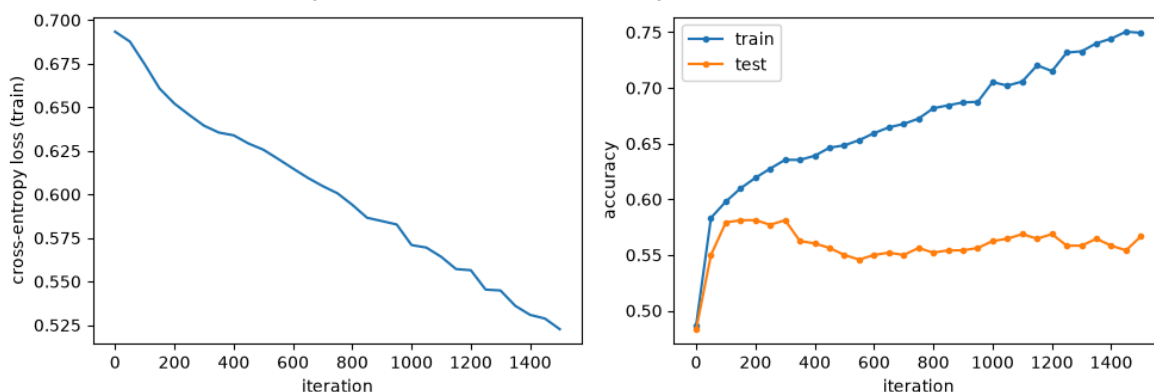
```
In [104... def two_layer_model(X, Y, n_h=128, learning_rate=0.05, num_iterations=150
                    X_test=None, Y_test=None, verbose=True):
    "Train LINEAR -> RELU -> LINEAR -> SIGMOID by full-batch gradient des
    n_x, n_y = X.shape[0], 1
    p = initialize_parameters(n_x, n_h, n_y)
    history = {"iteration": [], "cost": [], "train_acc": [], "test_acc": []}
    for i in range(num_iterations + 1):
        A2, cache = forward(X, p)
        if i % record_every == 0 or i == num_iterations:
            history["iteration"].append(i)
            history["cost"].append(compute_cost(A2, Y))
            history["train_acc"].append(accuracy(X, Y, p))
            if X_test is not None:
                history["test_acc"].append(accuracy(X_test, Y_test, p))
            if verbose and i % (5*record_every) == 0:
                print(f"iteration {i:5d} cost {history['cost'][-1]:.4f}
                      + (f" test acc {history['test_acc'][-1]:.3f}" if X
        if i == num_iterations:
            break
        grads = backward(p, cache, Y)
        learning_rate *= 0.999 # learning-rate sche
        p = update_parameters(p, grads, learning_rate)
    return p, pd.DataFrame(history)
```

```
In [105... np.random.seed(0)
n_x = IMAGE_SIZE * IMAGE_SIZE * 3 # 3072 inputs
params, hist = two_layer_model(Xf_train, Y_train, n_h=128, learning_rate=
                               X_test=Xf_test, Y_test=Y_test)

fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
ax[0].plot(hist.iteration, hist.cost)
ax[0].set_xlabel("iteration"); ax[0].set_ylabel("cross-entropy loss (train
ax[1].plot(hist.iteration, hist.train_acc, "o-", ms=3, label="train")
ax[1].plot(hist.iteration, hist.test_acc, "o-", ms=3, label="test")
ax[1].set_xlabel("iteration"); ax[1].set_ylabel("accuracy"); ax[1].legend
```

```
plt.tight_layout()
print(f"final train accuracy {hist.train_acc.iloc[-1]:.3f}    test accuracy {hist.test_acc.iloc[-1]:.3f}")
```

```
iteration    0  cost 0.6934  train acc 0.487  test acc 0.483
iteration   250  cost 0.6456  train acc 0.628  test acc 0.577
iteration   500  cost 0.6256  train acc 0.648  test acc 0.550
iteration   750  cost 0.6006  train acc 0.672  test acc 0.556
iteration  1000  cost 0.5709  train acc 0.705  test acc 0.562
iteration  1250  cost 0.5454  train acc 0.732  test acc 0.558
iteration  1500  cost 0.5226  train acc 0.749  test acc 0.567
final train accuracy 0.749    test accuracy 0.567
```



Read the two panels together. The training loss falls steadily and the training accuracy keeps rising, but the test accuracy levels off (or drops) well before the end. From that point on the network memorises the training images instead of learning what makes a dog a dog: it **overfits**. Chance level is 50 %, so the network did learn something, but a dense layer with 3072 inputs is a blunt tool for images. Part B shows a better one.

Task A1. Change the size of the hidden layer (for example 10, 50, 200 units) and the learning rate (for example 0.01, 0.05, 0.2). For each setting report the final train and test accuracy in a small table (a `pandas.DataFrame` is convenient). Use fewer iterations (say 600) to keep the run short. Which setting generalises best? Does a larger hidden layer help?

```
In [106... # Task A1: loop over a few (n_h, learning_rate) pairs, train with two_layer
# and collect the final train/test accuracies in a table.
# your code here
hidden_sizes = [10, 50, 200]
learning_rates = [0.01, 0.05, 0.2]
num_iterations = 600

results = []

# iterate through all combinations
for n_h in hidden_sizes: # n_h is the number of hidden neurons
    for lr in learning_rates: # lr is the number of different learning rates
        print(f"Training model with n_h={n_h}, learning_rate={lr}...")

        # gives return values
        parameters, costs = two_layer_model(
            Xf_train, Y_train, # this is the training set where the model
            n_h=n_h,
            learning_rate=lr,
            num_iterations=num_iterations,
```

```

X_test=Xf_test, # this is the test where we evaluate the model
Y_test=Y_test,
verbose=False) # this is added to mute the print statements

# evaluate predictions on train and test sets
train_preds = predict(Xf_train, parameters)
test_preds = predict(Xf_test, parameters)

# calculate accuracies
train_acc = (train_preds == Y_train).mean() * 100
test_acc = (test_preds == Y_test).mean() * 100

# store results
results.append({
    "Hidden Units (n_h)": n_h,
    "Learning Rate (lr)": lr,
    "Train Accuracy (%)": round(train_acc, 2), # is the model und
    "Test Accuracy (%)": round(test_acc, 2), # is the model accur
    "Generalization Gap (%)": round(train_acc - test_acc, 2)}) #

# create and display the summary DataFrame
results_df = pd.DataFrame(results)
display(results_df)

```

Training model with n_h=10, learning_rate=0.01...
 Training model with n_h=10, learning_rate=0.05...
 Training model with n_h=10, learning_rate=0.2...
 Training model with n_h=50, learning_rate=0.01...
 Training model with n_h=50, learning_rate=0.05...
 Training model with n_h=50, learning_rate=0.2...
 Training model with n_h=200, learning_rate=0.01...
 Training model with n_h=200, learning_rate=0.05...
 Training model with n_h=200, learning_rate=0.2...

	Hidden Units (n_h)	Learning Rate (lr)	Train Accuracy (%)	Test Accuracy (%)	Generalization Gap (%)
0	10	0.01	59.43	57.29	2.14
1	10	0.05	64.11	57.92	6.20
2	10	0.20	68.33	56.67	11.67
3	50	0.01	59.64	59.17	0.47
4	50	0.05	64.64	55.83	8.80
5	50	0.20	64.69	58.96	5.73
6	200	0.01	59.90	58.75	1.15
7	200	0.05	64.95	59.79	5.16
8	200	0.20	63.59	58.33	5.26

Question: Which setting generalises best? Does a larger hidden layer help?

Answer:

Row 7 ($n_h = 200$, $lr = 0.05$). It achieves the maximum test accuracy of 59.79% while keeping the generalization gap significantly lower (5.16%) compared to Row 4 (11.67%).

A larger hidden layer definitely seems to help! The models using 200 hidden units

have a higher test accuracy on average, likely because the extra nodes give the model more capacity to learn the features.

Task A2. Use `predict` on the test set, find the images that the network gets wrong, and show the first 12 of them in a grid with the predicted and the true class in the title. What do the misclassified pictures have in common?

```
In [ ]: # Task A2: find misclassified test images and show up to 12 of them (pred
names = {0: "bird", 1: "dog"}
# your code here

# get predictions on the test set using the trained parameters
test_preds = predict(Xf_test, parameters) # (assumes 'parameters' holds t

# flatten prediction array and true array for easy comparison
preds_flat = test_preds.flatten()
y_test_flat = Y_test.flatten()

# find indices where predictions do NOT equal true labels
misclassified_indices = np.where(preds_flat != y_test_flat)[0]

print(f"Total misclassified images on test set: {len(misclassified_indices)}")

# class labels mapping (0 = bird, 1 = dog)
class_names = {0: "bird", 1: "dog"}

# plot the first 12 misclassified images in a 3x4 grid
fig, axes = plt.subplots(3, 4, figsize=(12, 9))
axes = axes.flatten()

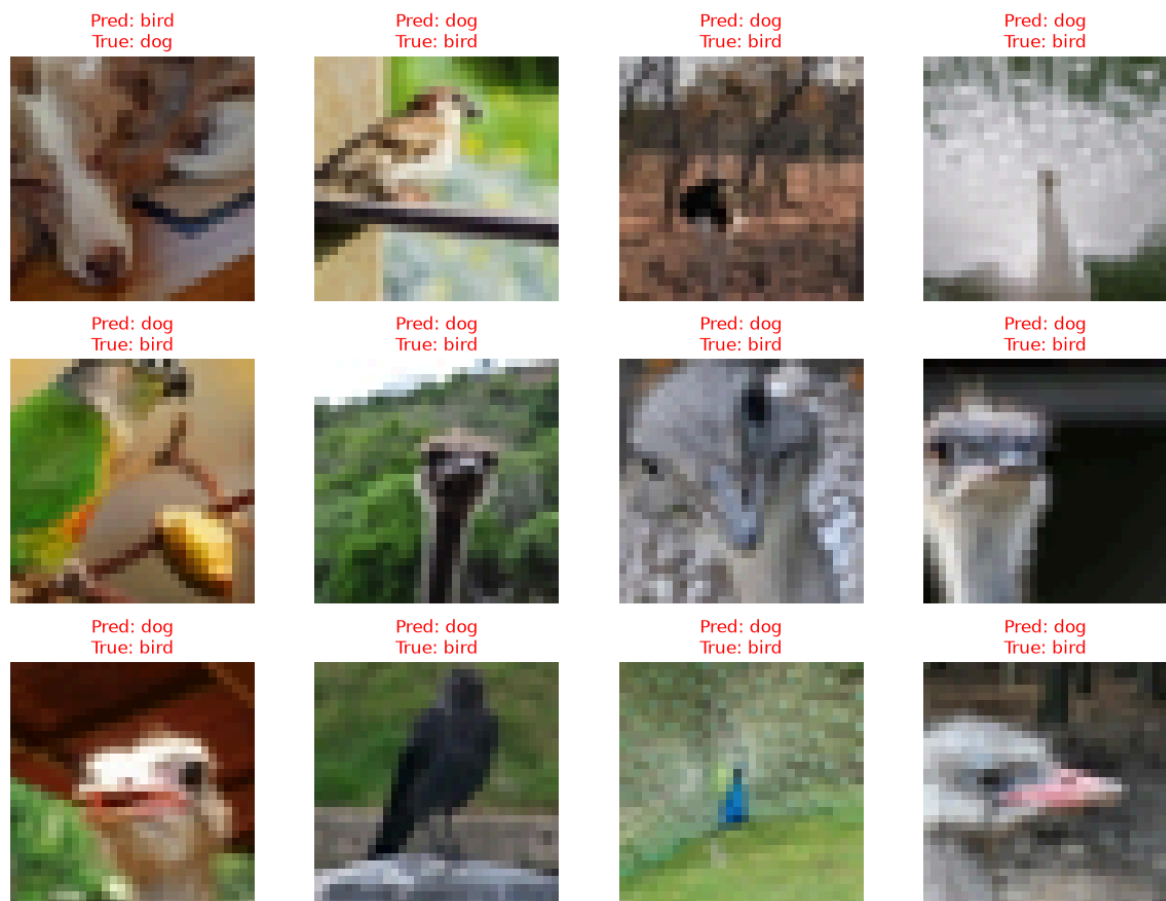
for i, idx in enumerate(misclassified_indices[:12]):
    # reshape flattened vector back into (32, 32, 3) image shape if neces
    # (or use X_test[idx] directly if unflattened array is available in m
    img = X_test[idx] if 'X_test' in globals() else Xf_test[:, idx].resha

    pred_label = class_names[preds_flat[idx]]
    true_label = class_names[y_test_flat[idx]]

    axes[i].imshow(img)
    axes[i].set_title(f"Pred: {pred_label}\nTrue: {true_label}", color="r")
    axes[i].axis("off")

plt.tight_layout()
plt.show()
```

Total misclassified images on test set: 190



Question: What do the misclassified pictures have in common?

Answer:

Examples of common biases include the background color (for instance, something green in the picture seems to increase the probability of a 'dog' prediction), images where only parts of the animal are visible, and ostrich heads, which also appear to introduce a strong bias

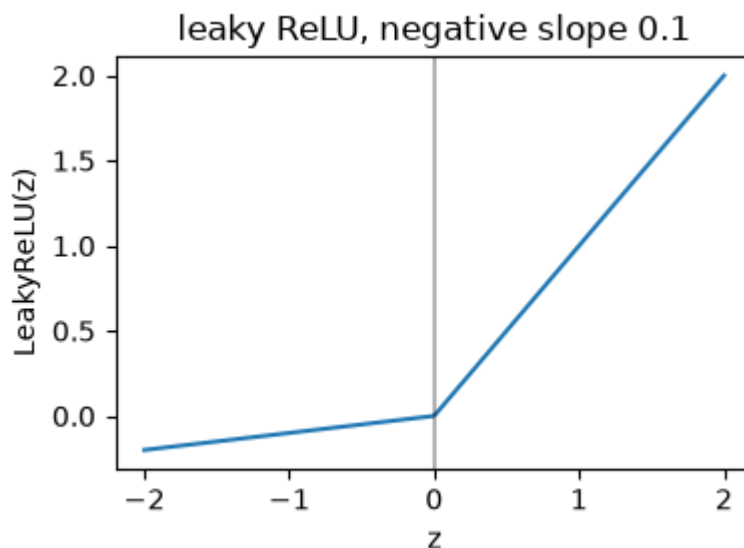
Part B. A convolutional network in Keras

A dense layer treats the 3072 pixel values as unrelated numbers. A **convolutional network** (CNN) instead slides a small filter (here 3×3 pixels) over the image and computes, at every position, the weighted sum of the pixels under the filter. One filter therefore has only $3 \cdot 3 \cdot 3 + 1 = 28$ parameters, yet it looks at the whole image. Each filter produces a *feature map*; a layer with 32 filters produces 32 maps. The building blocks are:

1. `layers.Conv2D(32, (3, 3), padding='same')` : 32 filters of 3×3; `padding='same'` keeps the image size.
2. An activation, here the *leaky ReLU*: like ReLU, but negative inputs are scaled by 0.1 instead of set to zero, so the gradient never vanishes entirely (plot below).
3. `layers.MaxPooling2D((2, 2))` : keeps the largest value in each 2×2 block, halving width and height. This reduces the number of parameters downstream and makes the network less sensitive to small shifts.

- After a few such blocks, `layers.Flatten()` turns the maps into a vector, and dense layers make the decision. The last layer is a single sigmoid neuron, exactly as in Part A.
- Keras trains the network with an optimizer (`Adam` , an improved gradient descent; Lecture 2) over several **epochs**. One epoch is one pass over the training set, in mini-batches of `batch_size` images; the weights are updated after every mini-batch.

```
In [ ]: z = np.linspace(-2, 2, 200)
fig, ax = plt.subplots(figsize=(4, 3))
ax.plot(z, np.where(z > 0, z, 0.1*z))
ax.axvline(0, color="k", alpha=0.3)
ax.set_xlabel("z"); ax.set_ylabel("LeakyReLU(z)"); ax.set_title("leaky Re
plt.tight_layout()
```



Data for Keras

Keras expects the images as an array of shape `(m, 32, 32, 3)`, which is how we loaded them. We reuse the split from Part A. Note the order of the splits: the **test set** was set aside first and is never used during training. Keras will additionally take 20 % of the *training* set as a **validation set** (`validation_split=0.2`); it monitors the loss on that part after every epoch. The validation loss guides decisions during training (when to stop), the test set gives the final, unbiased number.

```
In [ ]: print("train:", X_train.shape, y_train.shape, " test:", X_test.shape, y_
print("class balance in train: birds", np.sum(y_train == 0), " dogs", np.
```

```
train: (1920, 32, 32, 3) (1920,) test: (480, 32, 32, 3) (480,)
class balance in train: birds 960 dogs 960
```

Two models: without and with dropout

`build_cnn` returns the network. With `dropout=True` it inserts `Dropout` layers, which during training set a random fraction of the neurons to zero at every batch.

The network can then not rely on any single neuron and has to spread the evidence;

at test time all neurons are active. Dropout is a *regularisation* method (Lecture 2).
The layer names matter later in Part C.

```
In [ ]: def build_cnn(dropout=False, name="cnn"):
    m = keras.Sequential(name=name)
    m.add(layers.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3)))
    for k, filters in enumerate([32, 64, 128]):
        m.add(layers.Conv2D(filters, (3, 3), padding="same"))
        m.add(layers.LeakyReLU(negative_slope=0.1))
        m.add(layers.MaxPooling2D((2, 2)))
        if dropout:
            m.add(layers.Dropout(0.25 if k < 2 else 0.5))
    m.add(layers.Flatten(name="features"))
    m.add(layers.Dense(128))
    m.add(layers.LeakyReLU(negative_slope=0.1))
    if dropout:
        m.add(layers.Dropout(0.3))
    m.add(layers.Dense(1, activation="sigmoid"))
    m.compile(loss="binary_crossentropy", optimizer=keras.optimizers.Adam)
    return m

build_cnn().summary()
```

Model: "cnn"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 32, 32, 32)	
leaky_re_lu (LeakyReLU)	(None, 32, 32, 32)	
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	
conv2d_1 (Conv2D)	(None, 16, 16, 64)	
leaky_re_lu_1 (LeakyReLU)	(None, 16, 16, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	
conv2d_2 (Conv2D)	(None, 8, 8, 128)	
leaky_re_lu_2 (LeakyReLU)	(None, 8, 8, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	
features (Flatten)	(None, 2048)	
dense (Dense)	(None, 128)	
leaky_re_lu_3 (LeakyReLU)	(None, 128)	
dense_1 (Dense)	(None, 1)	

Total params: 355,649 (1.36 MB)

Trainable params: 355,649 (1.36 MB)

Non-trainable params: 0 (0.00 B)

Look at the parameter count. The three convolutional layers together hold fewer than 100 000 weights and process the whole image; the dense layer after **Flatten**

(2048 → 128) holds most of the rest. Compare this with Part A, where the first layer alone had $3072 \times 128 \approx 393\,000$ weights and saw nothing of the image structure.

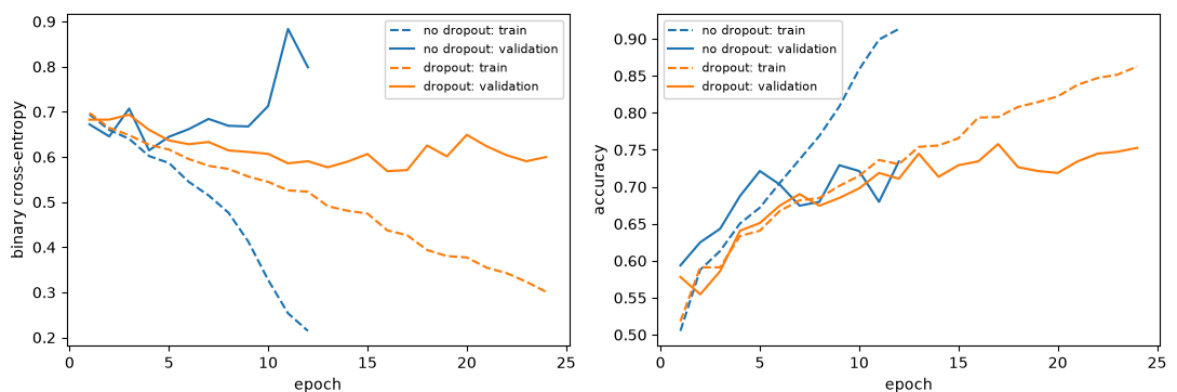
```
In [ ]: early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=8

histories, models, test_acc = {}, {}, {}
for label, use_dropout in [("no dropout", False), ("dropout", True)]:
    keras.utils.set_random_seed(0)
    model = build_cnn(dropout=use_dropout, name=label.replace(" ", "_"))
    h = model.fit(X_train, y_train, batch_size=50, epochs=50, validation_
                  callbacks=[early_stop], verbose=0)
    loss, acc = model.evaluate(X_test, y_test, verbose=0)
    histories[label], models[label], test_acc[label] = h.history, model,
    print(f"{label:11s}: stopped after {len(h.history['loss']):2d} epochs
          f"best val_loss {min(h.history['val_loss']):.3f}, test accuracy
          model.save(f"bird_dog_{label.replace(' ', '_')}.keras")
```

no dropout : stopped after 12 epochs, best val_loss 0.615, test accuracy 0.679

dropout : stopped after 24 epochs, best val_loss 0.568, test accuracy 0.723

```
In [ ]: fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
for label, c in [("no dropout", "C0"), ("dropout", "C1")]:
    h = histories[label]; ep = np.arange(1, len(h["loss"]) + 1)
    ax[0].plot(ep, h["loss"], "--", color=c, label=f"{label}: train")
    ax[0].plot(ep, h["val_loss"], "-", color=c, label=f"{label}: validation")
    ax[1].plot(ep, h["accuracy"], "--", color=c, label=f"{label}: train")
    ax[1].plot(ep, h["val_accuracy"], "-", color=c, label=f"{label}: validation")
ax[0].set_xlabel("epoch"); ax[0].set_ylabel("binary cross-entropy"); ax[0].legend()
ax[1].set_xlabel("epoch"); ax[1].set_ylabel("accuracy"); ax[1].legend()
plt.tight_layout()
```



Reading the curves. Without dropout the training loss drops towards zero within a few epochs while the validation loss reaches a minimum and then climbs again; the gap between the dashed and solid lines is the signature of overfitting. The

`EarlyStopping` callback watches `val_loss`, stops training when it has not improved for 8 epochs, and restores the weights from the best epoch. With dropout the training loss falls more slowly (part of the network is switched off at every step) and the validation loss follows it much longer, so the two curves stay close and the model trains for more epochs. The test accuracies printed above show which model generalises better; with only 1920 training images the difference is modest, and it varies from run to run.

Task B1. Modify `build_cnn` (copy it into the cell below under a new name) in one of two ways: add `layers.BatchNormalization()` after each `Conv2D`, or change the dropout rates (for example all 0.1, or all 0.5). Train with the same early-stopping callback, print the number of epochs and the test accuracy, and compare with the two models above. One sentence: what did the change do to the training and validation curves?

```
In [ ]: # Task B1: define a variant of build_cnn (BatchNormalization or other dro
# early stopping (validation_split=0.2, batch_size=50, epochs=50, verbose
# your code here

def build_cnn_new(dropout=False, name="cnn"):
    m = keras.Sequential(name=name)
    m.add(layers.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3)))
    for k, filters in enumerate([32, 64, 128]):
        m.add(layers.Conv2D(filters, (3, 3), padding="same"))
        m.add(layers.BatchNormalization()) # added
        m.add(layers.LeakyReLU(negative_slope=0.1))
        m.add(layers.MaxPooling2D((2, 2)))
        if dropout:
            m.add(layers.Dropout(0.25 if k < 2 else 0.5))
    m.add(layers.Flatten(name="features"))
    m.add(layers.Dense(128))
    m.add(layers.LeakyReLU(negative_slope=0.1))
    if dropout:
        m.add(layers.Dropout(0.3))
    m.add(layers.Dense(1, activation="sigmoid"))
    m.compile(loss="binary_crossentropy", optimizer=keras.optimizers.Adam)
    return m

build_cnn_new().summary()
```

Model: "cnn"

Layer (type)	Output Shape	
conv2d_27 (Conv2D)	(None, 32, 32, 32)	
batch_normalization_6 (BatchNormalization)	(None, 32, 32, 32)	
leaky_re_lu_36 (LeakyReLU)	(None, 32, 32, 32)	
max_pooling2d_27 (MaxPooling2D)	(None, 16, 16, 32)	
conv2d_28 (Conv2D)	(None, 16, 16, 64)	
batch_normalization_7 (BatchNormalization)	(None, 16, 16, 64)	
leaky_re_lu_37 (LeakyReLU)	(None, 16, 16, 64)	
max_pooling2d_28 (MaxPooling2D)	(None, 8, 8, 64)	
conv2d_29 (Conv2D)	(None, 8, 8, 128)	
batch_normalization_8 (BatchNormalization)	(None, 8, 8, 128)	
leaky_re_lu_38 (LeakyReLU)	(None, 8, 8, 128)	
max_pooling2d_29 (MaxPooling2D)	(None, 4, 4, 128)	
features (Flatten)	(None, 2048)	
dense_18 (Dense)	(None, 128)	
leaky_re_lu_39 (LeakyReLU)	(None, 128)	
dense_19 (Dense)	(None, 1)	

Total params: 356,545 (1.36 MB)

Trainable params: 356,097 (1.36 MB)

Non-trainable params: 448 (1.75 KB)

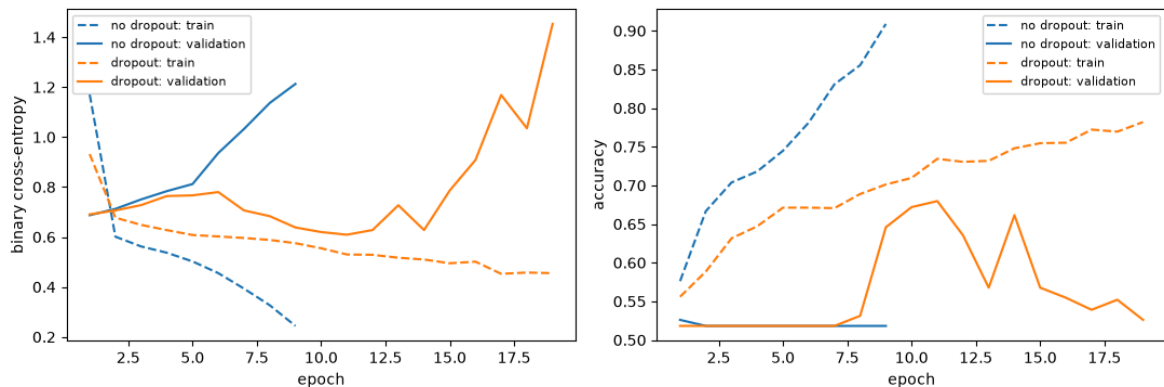
```
In [ ]: early_stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=8

histories, models, test_acc = {}, {}, {}
for label, use_dropout in [("no dropout", False), ("dropout", True)]:
    keras.utils.set_random_seed(0)
    model = build_cnn_new(dropout=use_dropout, name=label.replace(" ", "_"))
    h = model.fit(X_train, y_train, batch_size=50, epochs=50, validation_data=(X_test, y_test),
                  callbacks=[early_stop], verbose=0)
    loss, acc = model.evaluate(X_test, y_test, verbose=0)
    histories[label], models[label], test_acc[label] = h.history, model, acc
    print(f"{label:11s}: stopped after {len(h.history['loss']):2d} epochs, best val_loss {min(h.history['val_loss']):.3f}, test accuracy {acc:.3f}")
    model.save(f"bird_dog_{label.replace(' ', '_')}.keras")
```

no dropout : stopped after 9 epochs, best val_loss 0.687, test accuracy 0.552

dropout : stopped after 19 epochs, best val_loss 0.609, test accuracy 0.648

```
In [ ]: fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
for label, c in [("no dropout", "C0"), ("dropout", "C1")]:
    h = histories[label]; ep = np.arange(1, len(h["loss"]) + 1)
    ax[0].plot(ep, h["loss"], "--", color=c, label=f"{label}: train")
    ax[0].plot(ep, h["val_loss"], "-", color=c, label=f"{label}: validation")
    ax[1].plot(ep, h["accuracy"], "--", color=c, label=f"{label}: train")
    ax[1].plot(ep, h["val_accuracy"], "-", color=c, label=f"{label}: validation")
ax[0].set_xlabel("epoch"); ax[0].set_ylabel("binary cross-entropy"); ax[0].legend()
ax[1].set_xlabel("epoch"); ax[1].set_ylabel("accuracy"); ax[1].legend()
plt.tight_layout()
```



Question: What did the Batch Normalization do to the training and validation curves?

Answer:

It seems to reduce the total number of epochs needed for both models. For the (no dropouts) model severe overfitting is preserved in both cases, for the (dropouts) model validation curves are fluctuating a lot.

Task B2. Take the dropout model from `models["dropout"]`, predict the probabilities for the test set (`model.predict(X_test, verbose=0)`), and show 12 misclassified images with the predicted probability of "dog" in the title. Are the mistakes confident (probability near 0 or 1) or borderline (near 0.5)?

```
In [ ]: # Task B2: predicted probabilities on the test set, grid of 12 misclassified
# your code here

# get predicted probabilities from the trained Keras dropout model
model_dropout = models["dropout"]
probs = model_dropout.predict(X_test, verbose=0).flatten()
# convert probabilities (> 0.5) to binary predictions (0 = bird, 1 = dog)
preds = (probs > 0.5).astype(int)
y_true = y_test.flatten()
# find misclassified indices
misclassified_indices = np.where(preds != y_true)[0]
print(f"Total misclassified images on test set: {len(misclassified_indices)}")
# plot the first 12 misclassified images in a 3x4 grid
fig, axes = plt.subplots(3, 4, figsize=(12, 9))
axes = axes.flatten()

for i, idx in enumerate(misclassified_indices[:12]):
    img = X_test[idx]
    p_dog = probs[idx]
    true_class = names[y_true[idx]]
    pred_class = names[preds[idx]]
```

```

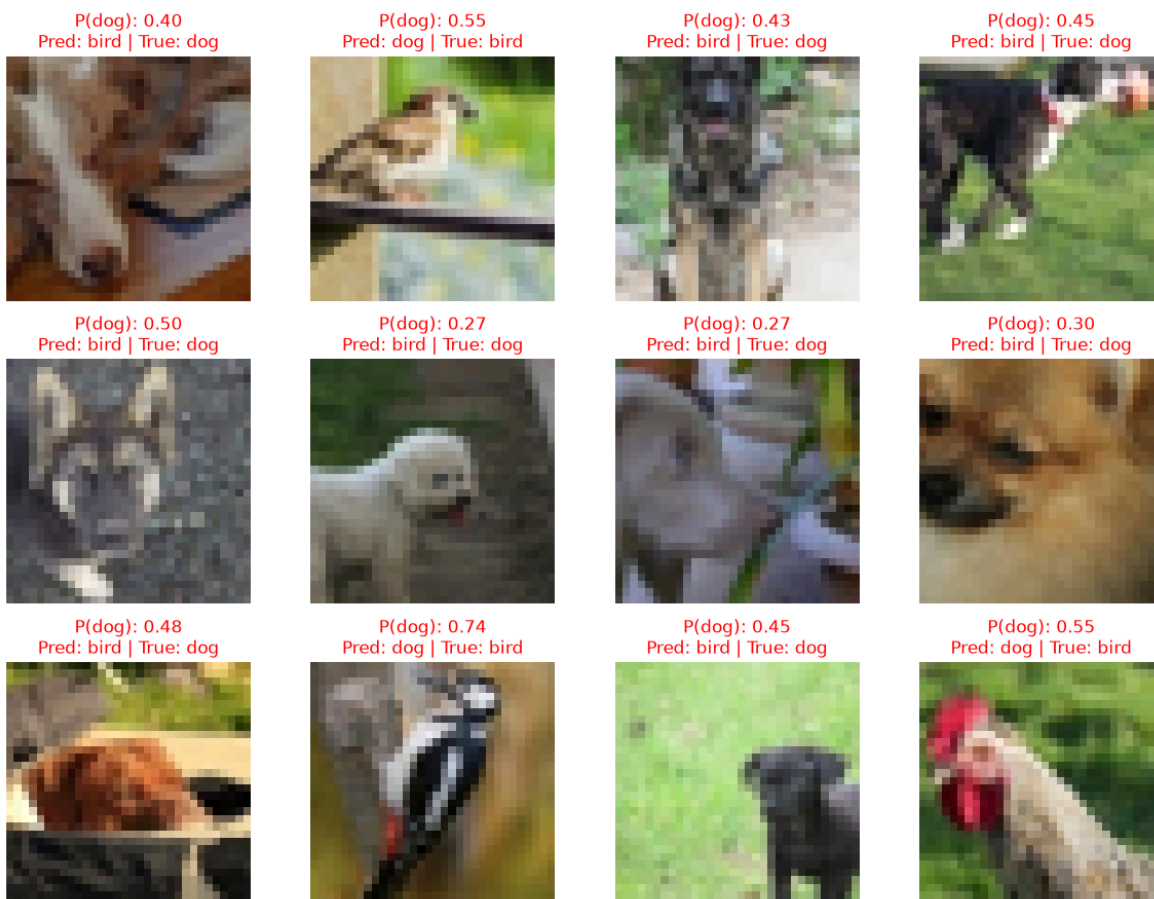
axes[i].imshow(img)
# display predicted dog probability, predicted class, and true class
axes[i].set_title(f"P(dog): {p_dog:.2f}\nPred: {pred_class} | True: {true_class}")
axes[i].axis("off")

# hide unused subplots if fewer than 12 images are misclassified
for j in range(i + 1, len(axes)):
    axes[j].axis("off")

plt.tight_layout()
plt.show()

```

Total misclassified images on test set: 169 out of: 480



Question: Are the mistakes confident (probability near 0 or 1) or borderline (near 0.5)?

Answer:

Based on the 12 misclassified images it seems that the majority of mistakes happens close to the borderline (50% limit).

Part C. A small data set: 19 faces

The folder `faces` contains 19 photos of your teacher, some smiling, some not. Old versions of this lab trained a CNN from scratch on them. With 19 images that cannot work in a meaningful way: a CNN has thousands of parameters and will memorise 19 pictures perfectly while learning nothing you can trust. The honest procedure is

1. use **cross-validation**, so that every prediction is made for an image the model did not see,
2. report the accuracy **with its uncertainty**, and
3. bring in knowledge from a larger data set (**transfer learning**, Task C1).

We crop the face region as in the original notebook, average the colour channels to grey, and resize to 32×32. The labels (1 = happy) are given in the order of the sorted file names.

```
In [ ]: face_files = sorted((DATA / "faces").glob("*.jpg")) # Load the data and p
happy = np.array([0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0])
assert len(face_files) == len(happy) == 19

def load_face(p, size=IMAGE_SIZE):
    img = np.asarray(Image.open(p).convert("RGB")[200:600, 500:800, :])
    return np.asarray(Image.fromarray(img).resize((size, size)), dtype=np

faces_rgb = np.array([load_face(p) for p in face_files]) # (19, 32
faces_gray = faces_rgb.mean(axis=3) # (19, 32
print("faces:", faces_gray.shape, " happy:", happy.sum(), " not happy:",

fig, ax = plt.subplots(2, 10, figsize=(12, 3))
for a, img, lab, p in zip(ax.ravel(), faces_gray, happy, face_files):
    a.imshow(img, cmap="gray"); a.axis("off"); a.set_title(("happy" if la
ax.ravel()[-1].axis("off")
plt.tight_layout() # plots all the 19 faces :) or :(
```

faces: (19, 32, 32) happy: 7 not happy: 12



Cross-validation and uncertainty

The majority class ("not happy", 12 of 19) already gives 63 % accuracy, so that is the number to beat. We use 5-fold stratified cross-validation: the model is trained on four fifths of the images and tested on the remaining fifth, five times, so every image is predicted once while held out.

For a classifier with true accuracy p tested on n images, the standard error of the measured accuracy is $\sqrt{p(1-p)/n}$. With $n = 19$ that is about 0.1: an accuracy of 0.8 means "somewhere between 0.6 and 1.0". Keep that in mind when you read the numbers.

We compare two models: logistic regression on the 1024 grey pixel values, and a tiny CNN.

```
In [ ]: from sklearn.model_selection import StratifiedKFold
from sklearn.linear_model import LogisticRegression
```

```

def cv_accuracy(fit_predict, X, y, n_splits=5, seed=0): # Cross-Validation
    "Cross-validated accuracy and its binomial standard error. fit_predict
    pred = np.zeros_like(y)
    for tr, te in StratifiedKFold(n_splits, shuffle=True, random_state=seed):
        pred[te] = fit_predict(X[tr], y[tr], X[te])
    acc = np.mean(pred == y)
    return acc, np.sqrt(acc*(1 - acc)/len(y)), pred

def logreg_fit_predict(Xtr, ytr, Xte): # Logistic Regression Model
    clf = LogisticRegression(C=0.1, max_iter=2000).fit(Xtr.reshape(len(Xtr)), ytr)
    return clf.predict(Xte.reshape(len(Xte), -1))

def tiny_cnn_fit_predict(Xtr, ytr, Xte): # Tiny CNN (Deep Learning) Model
    keras.utils.set_random_seed(0)
    m = keras.Sequential([layers.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 1)),
                          layers.Conv2D(8, (3, 3), padding="same", activation="relu"),
                          layers.MaxPooling2D((4, 4)),
                          layers.Flatten(),
                          layers.Dense(1, activation="sigmoid")])
    m.compile(loss="binary_crossentropy", optimizer=keras.optimizers.Adam)
    m.fit(Xtr[...], ytr, epochs=60, batch_size=8, verbose=0)
    return (m.predict(Xte[...], verbose=0).ravel() > 0.5).astype(int)

acc_lr, se_lr, _ = cv_accuracy(logreg_fit_predict, faces_gray, happy)
acc_cnn, se_cnn, _ = cv_accuracy(tiny_cnn_fit_predict, faces_gray, happy)
print(f"majority class      : {max(happy.mean(), 1 - happy.mean()):.2f}")
print(f"logistic regression (CV): {acc_lr:.2f} ± {se_lr:.2f}")
print(f"tiny CNN              (CV): {acc_cnn:.2f} ± {se_cnn:.2f}")

```

```

majority class      : 0.63
logistic regression (CV): 0.63 ± 0.11
tiny CNN              (CV): 0.63 ± 0.11

```

Task C1 (transfer learning). The bird/dog CNN has learned filters that respond to edges, blobs and textures. Those are useful for any image. Build a feature extractor from the dropout model, `keras.Model(inputs=cnn.input, outputs=cnn.get_layer("features").output)`, run the 19 RGB face crops (`faces_rgb`) through it to get a 2048-number feature vector per image, and fit a `LogisticRegression` on these features with the same 5-fold `cv_accuracy`. Compare with the two results above. Is the improvement larger than the uncertainty?

```

In [78]: # extract the dropout model instance from the models dictionary
dropout_model = models["dropout"]

# build the feature extractor using that trained model instance
feature_extractor = keras.Model(inputs=dropout_model.inputs, outputs=dropout_model.get_layer("features").output)

# extract the 2048-length feature vectors for the 19 face images
X_faces_features = feature_extractor.predict(faces_rgb)

# fit Logistic Regression with 5-fold CV
log_reg = LogisticRegression(max_iter=1000)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)

cv_scores = cross_val_score(log_reg, X_faces_features, happy, cv=cv)
mean_acc = cv_scores.mean()
std_acc = cv_scores.std()

```

```
print(f"Transfer Learning CV Accuracy: {mean_acc:.3f} ± {std_acc:.3f}")
```

```
1/1 ————— 0s 53ms/step
```

```
1/1 ————— 0s 53ms/step
```

```
Transfer Learning CV Accuracy: 0.883 ± 0.145
```

Question: Compare with the two results above. Is the improvement larger than the uncertainty?

Answer:

Improvement: $0.883 - 0.63 = 0.253$

Uncertainty 1: 0.11

Uncertainty 2: 0.145

The improvement is larger than the uncertainty.

Where does the model look? (demo)

An *occlusion map* asks a simple question: if we cover a small patch of the image with grey, how much does the predicted probability of "happy" change? Patches whose covering matters most are where the model looks. This needs only `predict`, so it works on any backend. We use the transfer-learning pipeline fitted on all 19 images. (A proper saliency map uses the gradient of the output with respect to the pixels; that is a Lecture 2 topic.)

```
In [80]: face_features = X_faces_features
clf_all = LogisticRegression(C=0.1, max_iter=5000).fit(face_features, happy)
i, patch, stride = 1, 6, 2 # image 02 is
img = faces_rgb[i]
positions = [(r, c) for r in range(0, IMAGE_SIZE - patch + 1, stride)
              for c in range(0, IMAGE_SIZE - patch + 1, stride)]
occluded = np.repeat(img[None], len(positions), axis=0)
for k, (r, c) in enumerate(positions):
    occluded[k, r:r+patch, c:c+patch, :] = 0.5
p_full = clf_all.predict_proba(feature_extractor.predict(img[None]))
p_occ = clf_all.predict_proba(feature_extractor.predict(occluded))
n = int(np.sqrt(len(positions)))
heat = (p_full - p_occ).reshape(n, n)

fig, ax = plt.subplots(1, 2, figsize=(7, 3.2))
ax[0].imshow(img); ax[0].set_title(f"p(happy) = {p_full:.2f}"); ax[0].axis('off')
im = ax[1].imshow(heat, cmap="coolwarm", vmin=-np.abs(heat).max(), vmax=np.abs(heat).max())
ax[1].set_title("drop in p(happy) when patch covered"); ax[1].axis("off")
plt.colorbar(im, ax=ax[1], fraction=0.046)
plt.tight_layout()
```



Part D. MNIST handwritten digits

MNIST is the classic benchmark: 70 000 grey images of 28×28 pixels showing the digits 0–9. Keras downloads it (11 MB). To keep the run short we train on a 10 000-image subset and test on the full 10 000-image test set. Ten classes need two changes compared with Part B: the last layer has 10 neurons with `softmax` activation (one probability per digit), and the loss is `sparse_categorical_crossentropy`, which takes the integer labels directly.

```
In [81]: (x_train_full, y_train_full), (x_test_d, y_test_d) = keras.datasets.mnist
print("full training set:", x_train_full.shape, y_train_full.shape, " tes
      " pixel range:", x_train_full.min(), "-", x_train_full.max())

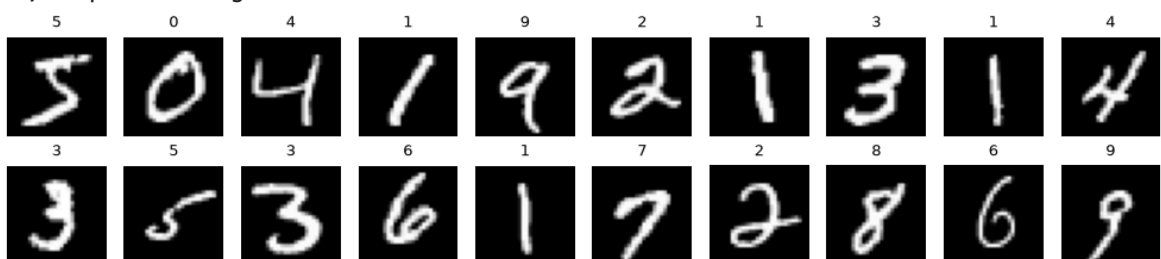
N_TRAIN = 10_000
x_train_d = x_train_full[:N_TRAIN].astype("float32")[:, :, :, None] / 255.0
y_train_d = y_train_full[:N_TRAIN]
x_test_d = x_test_d.astype("float32")[:, :, :, None] / 255.0

fig, ax = plt.subplots(2, 10, figsize=(10, 2.4))
for k, a in enumerate(ax.ravel()):
    a.imshow(x_train_d[k, :, :, 0], cmap="gray"); a.set_title(int(y_train
plt.tight_layout()
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 2s 0us/step

full training set: (60000, 28, 28) (60000,) test: (10000, 28, 28) (10000,) pixel range: 0 – 255



Task D1. Build and train a small CNN for MNIST. Suggested layout: `Input((28, 28, 1))`, two blocks of `Conv2D` (16 and 32 filters, 3×3, `activation='relu'`)

each followed by `MaxPooling2D((2, 2))`, then `Flatten`, `Dense(64, activation='relu')`, `Dropout(0.3)` and `Dense(10, activation='softmax')`. Compile with `loss='sparse_categorical_crossentropy'`, optimizer `adam`, `metrics=['accuracy']`. Train for 4 epochs with `batch_size=64` and `validation_split=0.1`, then evaluate on the test set and print the test accuracy. Store the model in `mnist_model`, the next cell uses it.

```
In [93]: # Task D1: build, compile, train (4 epochs) and evaluate a CNN for MNIST.
keras.utils.set_random_seed(0)
mnist_model = keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    # your code here
    # 1th Conv block
    layers.Conv2D(16, (3, 3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    # 2th Conv block
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(10, activation='softmax')])

# your code here (compile, fit, evaluate)
# Compile model
mnist_model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

# Train model
mnist_model.fit(
    x_train_d, y_train_d,
    epochs=4,
    batch_size=64,
    validation_split=0.1,
)

test_loss, test_acc = mnist_model.evaluate(x_test_d, y_test_d, verbose=0)
print(f"MNIST Test Accuracy: {test_acc:.4f}")
```

Epoch 1/4

141/141 ————— **1s** 3ms/step – accuracy: 0.7184 – loss: 0.9151
– val_accuracy: 0.9310 – val_loss: 0.2496

Epoch 2/4

141/141 ————— **0s** 3ms/step – accuracy: 0.9106 – loss: 0.2960
– val_accuracy: 0.9570 – val_loss: 0.1537

Epoch 3/4

141/141 ————— **0s** 3ms/step – accuracy: 0.9366 – loss: 0.2138
– val_accuracy: 0.9580 – val_loss: 0.1414

Epoch 4/4

141/141 ————— **0s** 3ms/step – accuracy: 0.9516 – loss: 0.1642
– val_accuracy: 0.9640 – val_loss: 0.1110

MNIST Test Accuracy: 0.9686

The next cell evaluates `mnist_model`: it prints the confusion matrix (rows: true digit, columns: predicted digit) and shows some of the digits the network gets wrong, with the prediction and the true label.

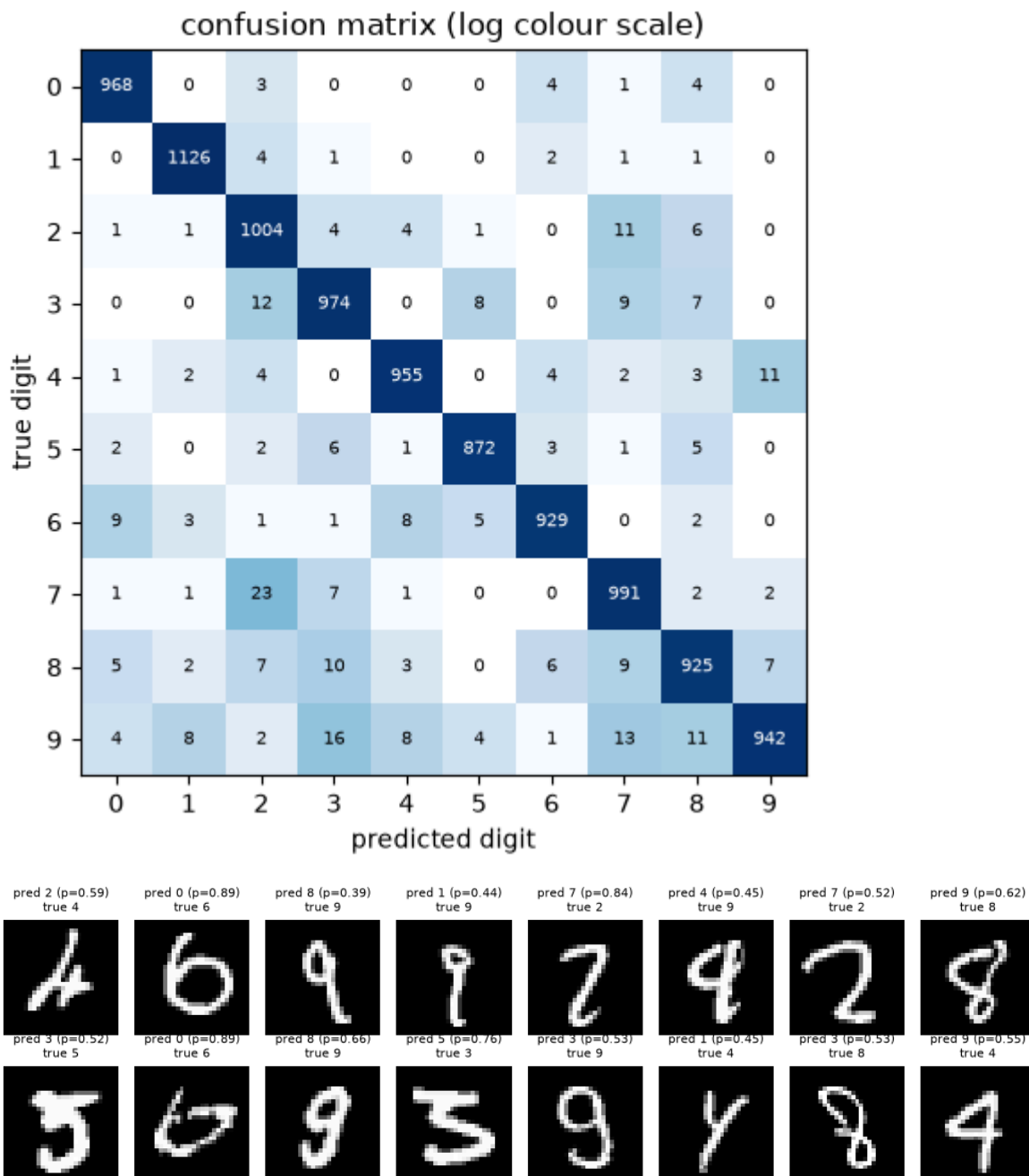
```
In [94]: from sklearn.metrics import confusion_matrix
from matplotlib.colors import LogNorm
prob_d = mnist_model.predict(x_test_d, verbose=0)
pred_d = prob_d.argmax(axis=1)
cm = confusion_matrix(y_test_d, pred_d)

fig, ax = plt.subplots(figsize=(5.5, 5))
ax.imshow(cm, cmap="Blues", norm=LogNorm())
for r in range(10):
    for c in range(10):
        ax.text(c, r, cm[r, c], ha="center", va="center", fontsize=7, color="black")
ax.set_xticks(range(10)); ax.set_yticks(range(10))
ax.set_xlabel("predicted digit"); ax.set_ylabel("true digit"); ax.set_title("Confusion Matrix")
plt.tight_layout()

wrong = np.where(pred_d != y_test_d)[0]
fig, ax = plt.subplots(2, 8, figsize=(10, 3))
for a, i in zip(ax.ravel(), wrong[:16]):
    a.imshow(x_test_d[i, :, :, 0], cmap="gray"); a.axis("off")
    a.set_title(f"pred {pred_d[i]} (p={prob_d[i].max():.2f})\ntrue {y_test_d[i]}")
plt.tight_layout()

off = cm.copy(); np.fill_diagonal(off, 0)
pairs = sorted(((off[r, c], r, c) for r in range(10) for c in range(10) if off[r, c] > 0))
print("most frequent confusions (count, true -> predicted):", [(int(n), r, c) for (n, r, c) in pairs])
```

most frequent confusions (count, true -> predicted): [(23, 7, 2), (16, 9, 3), (13, 9, 7), (12, 3, 2), (11, 9, 8), (11, 4, 9)]



Task D2. From the confusion matrix and the list above: which digit pairs are confused most often, and why might that be? Write two or three sentences in the cell below. Then look at the misclassified examples: would you have read them correctly?

Your answer here.

The most common mix-up (23 instances) happens when the true digit is a 7, but the model predicts a 2. This occurs because the two digits look remarkably similar when written quickly (especially depending on how sloped the top line or stroke is). While I like to think I could tell them apart a bit better than the model, it is an understandable mistake given how much visual structure they share!

What to take to Lecture 2

- **Overfitting** shows up as a growing gap between training and validation (or test) curves: the training loss keeps falling while the validation loss turns upward. You

saw it in Part A (accuracy panel) and in Part B (no-dropout model).

- **Parameter counts** matter. A dense layer on raw pixels needs 3072×128 weights and learns nothing about neighbouring pixels; a 3×3 convolution needs 28 weights per filter and sees the whole image.
- **Learning-rate decay** (the factor 0.999 in Part A) is the simplest learning-rate schedule. Lecture 2 covers momentum, Adam and schedules in more detail.
- **Early stopping** and **dropout** are regularisation methods: they limit how far the network can memorise the training set. Lecture 2 adds weight decay, data augmentation and batch normalisation.
- **Small data sets** call for cross-validation, an error bar, and transfer learning rather than a bigger network.